
NOIP 2026 模拟赛题解

Ehundategh

Gioush 大队

2026 年 13 月 32 日

目录

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间

Contents

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间

数据点 1 ~ 3 15 Pts

- ▶ 这一档保证 $n \leq 10$ 。可以枚举购买的糖果集合，再枚举其中哪些糖果被装入礼盒。

数据点 1 ~ 3 15 Pts

- ▶ 这一档保证 $n \leq 10$ 。可以枚举购买的糖果集合，再枚举其中哪些糖果被装入礼盒。
- ▶ 对于一个还没有结算的糖果，可以单独购买，也可以选择另一颗尚未结算的糖果与它装入同一个礼盒。

数据点 1 ~ 3 15 Pts

- ▶ 这一档保证 $n \leq 10$ 。可以枚举购买的糖果集合，再枚举其中哪些糖果被装入礼盒。
- ▶ 对于一个还没有结算的糖果，可以单独购买，也可以选择另一颗尚未结算的糖果与它装入同一个礼盒。
- ▶ 检查总费用是否不超过 m ，取合法方案中购买数量的最大值。

数据点 1 ~ 3 15 Pts

- ▶ 这一档保证 $n \leq 10$ 。可以枚举购买的糖果集合，再枚举其中哪些糖果被装入礼盒。
- ▶ 对于一个还没有结算的糖果，可以单独购买，也可以选择另一颗尚未结算的糖果与它装入同一个礼盒。
- ▶ 检查总费用是否不超过 m ，取合法方案中购买数量的最大值。
- ▶ 时间复杂度可以做到 $\mathcal{O}(3^n n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 4 ~ 6 30 Pts

- ▶ 这一档保证 $k = 0$ ，也就是不能使用礼盒。

数据点 4 ~ 6 30 Pts

- ▶ 这一档保证 $k = 0$ ，也就是不能使用礼盒。
- ▶ 将所有价格从小到大排序。若要购买 x 颗糖果，选择价格最小的 x 颗一定不劣。

数据点 4 ~ 6 30 Pts

- ▶ 这一档保证 $k = 0$ ，也就是不能使用礼盒。
- ▶ 将所有价格从小到大排序。若要购买 x 颗糖果，选择价格最小的 x 颗一定不劣。
- ▶ 维护价格前缀和，依次判断每个 x 是否满足

$$\sum_{i=1}^x a_i \leq m.$$

数据点 4 ~ 6 30 Pts

- ▶ 这一档保证 $k = 0$ ，也就是不能使用礼盒。
- ▶ 将所有价格从小到大排序。若要购买 x 颗糖果，选择价格最小的 x 颗一定不劣。
- ▶ 维护价格前缀和，依次判断每个 x 是否满足

$$\sum_{i=1}^x a_i \leq m.$$

- ▶ 排序后线性扫描即可，时间复杂度为 $\mathcal{O}(n \log n)$ 。

数据点 7 ~ 9 45 Pts

- ▶ 这一档保证 $k \leq 1$ 。仍然固定购买数量 x ，并选择价格最小的 x 颗糖果。

数据点 7 ~ 9 45 Pts

- ▶ 这一档保证 $k \leq 1$ 。仍然固定购买数量 x ，并选择价格最小的 x 颗糖果。
- ▶ 若 $x \geq 2$ 且可以使用礼盒，将其中最贵的两颗糖果装入礼盒，一定能够省去较便宜的那颗糖果的价格。

数据点 7 ~ 9 45 Pts

- ▶ 这一档保证 $k \leq 1$ 。仍然固定购买数量 x ，并选择价格最小的 x 颗糖果。
- ▶ 若 $x \geq 2$ 且可以使用礼盒，将其中最贵的两颗糖果装入礼盒，一定能够省去较便宜的那颗糖果的价格。
- ▶ 因而最小费用为

$$\sum_{i=1}^x a_i - a_{x-1}.$$

当 $x < 2$ 或 $k = 0$ 时，仍为普通前缀和。

数据点 7 ~ 9 45 Pts

- ▶ 这一档保证 $k \leq 1$ 。仍然固定购买数量 x ，并选择价格最小的 x 颗糖果。
- ▶ 若 $x \geq 2$ 且可以使用礼盒，将其中最贵的两颗糖果装入礼盒，一定能够省去较便宜的那颗糖果的价格。
- ▶ 因而最小费用为

$$\sum_{i=1}^x a_i - a_{x-1}.$$

当 $x < 2$ 或 $k = 0$ 时，仍为普通前缀和。

- ▶ 这一档已经说明，固定购买集合以后，应优先把较贵的糖果放入礼盒。

数据点 10 ~ 15 75 Pts

► 对于固定的 x , 令

$$r = \min\left(k, \left\lfloor \frac{x}{2} \right\rfloor\right).$$

因为价格均为正数, 使用满 r 个礼盒一定不劣。

数据点 10 ~ 15 75 Pts

- 对于固定的 x , 令

$$r = \min\left(k, \left\lfloor \frac{x}{2} \right\rfloor\right).$$

因为价格均为正数, 使用满 r 个礼盒一定不劣。

- 选择最贵的 $2r$ 颗糖果装入礼盒。将它们从小到大相邻配对, 每一对只需要支付较贵者的价格。

数据点 10 ~ 15 75 Pts

- ▶ 对于固定的 x ，令

$$r = \min\left(k, \left\lfloor \frac{x}{2} \right\rfloor\right).$$

因为价格均为正数，使用满 r 个礼盒一定不劣。

- ▶ 选择最贵的 $2r$ 颗糖果装入礼盒。将它们从小到大相邻配对，每一对只需要支付较贵者的价格。
- ▶ 直接枚举 x ，再用 $\mathcal{O}(x)$ 的时间计算费用，总时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 10 ~ 15 75 Pts

- ▶ 对于固定的 x ，令

$$r = \min\left(k, \left\lfloor \frac{x}{2} \right\rfloor\right).$$

因为价格均为正数，使用满 r 个礼盒一定不劣。

- ▶ 选择最贵的 $2r$ 颗糖果装入礼盒。将它们从小到大相邻配对，每一对只需要支付较贵者的价格。
- ▶ 直接枚举 x ，再用 $\mathcal{O}(x)$ 的时间计算费用，总时间复杂度为 $\mathcal{O}(n^2)$ 。
- ▶ 当前瓶颈只剩下重复计算相邻配对中较贵位置的价格之和。

正解 100 Pts

- 先证明固定购买数量 x 时，只考虑最便宜的 x 颗糖果即可。

正解 100 Pts

- ▶ 先证明固定购买数量 x 时，只考虑最便宜的 x 颗糖果即可。
- ▶ 任意购买方案中的价格从小到大为 $c_1, c_2, \dots, c_x$ 。全体糖果排序后有 $a_i \leq c_i$ 。

正解 100 Pts

- ▶ 先证明固定购买数量 x 时，只考虑最便宜的 x 颗糖果即可。
- ▶ 任意购买方案中的价格从小到大为 $c_1, c_2, \dots, c_x$ 。全体糖果排序后有 $a_i \leq c_i$ 。
- ▶ 单独结算与礼盒结算都只会取某个价格或两个价格的最大值，因此把每个 c_i 换成不大的 a_i 不会增加费用。

正解 100 Pts

- ▶ 先证明固定购买数量 x 时，只考虑最便宜的 x 颗糖果即可。
- ▶ 任意购买方案中的价格从小到大为 $c_1, c_2, \dots, c_x$ 。全体糖果排序后有 $a_i \leq c_i$ 。
- ▶ 单独结算与礼盒结算都只会取某个价格或两个价格的最大值，因此把每个 c_i 换成不大的 a_i 不会增加费用。
- ▶ 所以存在一个最优方案，购买的恰好是排序后的前 x 颗糖果。

正解 100 Pts

- ▶ 现在考虑礼盒中的糖果。若某颗较便宜的糖果在礼盒中，而更贵的糖果单独结算，交换二者不会增加费用。

正解 100 Pts

- ▶ 现在考虑礼盒中的糖果。若某颗较便宜的糖果在礼盒中，而更贵的糖果单独结算，交换二者不会增加费用。
- ▶ 因此可以让最贵的 $2r$ 颗糖果全部进入礼盒。

正解 100 Pts

- ▶ 现在考虑礼盒中的糖果。若某颗较便宜的糖果在礼盒中，而更贵的糖果单独结算，交换二者不会增加费用。
- ▶ 因此可以让最贵的 $2r$ 颗糖果全部进入礼盒。
- ▶ 对于 $a \leq b \leq c \leq d$ ，相邻配对的费用为 $b + d$ ，而另外两种配对的费用分别为 $c + d$ 与 $c + d$ 。

正解 100 Pts

- ▶ 现在考虑礼盒中的糖果。若某颗较便宜的糖果在礼盒中，而更贵的糖果单独结算，交换二者不会增加费用。
- ▶ 因此可以让最贵的 $2r$ 颗糖果全部进入礼盒。
- ▶ 对于 $a \leq b \leq c \leq d$ ，相邻配对的费用为 $b + d$ ，而另外两种配对的费用分别为 $c + d$ 与 $c + d$ 。
- ▶ 不断消去最小的四颗待配对糖果，可以得到：排序后相邻配对最优。

正解 100 Pts

令 $L = x - 2r$ 。前 L 颗糖果单独结算，后 $2r$ 颗糖果相邻配对，因此

$$\text{Cost}(x) = \sum_{i=1}^L a_i + \sum_{j=1}^r a_{L+2j}.$$

► 第二项只会取区间 $(L, x]$ 中与 x 奇偶性相同的位置。

正解 100 Pts

令 $L = x - 2r$ 。前 L 颗糖果单独结算，后 $2r$ 颗糖果相邻配对，因此

$$\text{Cost}(x) = \sum_{i=1}^L a_i + \sum_{j=1}^r a_{L+2j}.$$

- ▶ 第二项只会取区间 $(L, x]$ 中与 x 奇偶性相同的位置。
- ▶ 分别维护普通前缀和 Pre、奇数位置和 Odd、偶数位置和 Even。

正解 100 Pts

令 $L = x - 2r$ 。前 L 颗糖果单独结算，后 $2r$ 颗糖果相邻配对，因此

$$\text{Cost}(x) = \sum_{i=1}^L a_i + \sum_{j=1}^r a_{L+2j}.$$

- ▶ 第二项只会取区间 $(L, x]$ 中与 x 奇偶性相同的位置。
- ▶ 分别维护普通前缀和 Pre、奇数位置和 Odd、偶数位置和 Even。
- ▶ 每个 x 的费用可以在 $\mathcal{O}(1)$ 时间内求出。

正解 100 Pts

令 $L = x - 2r$ 。前 L 颗糖果单独结算，后 $2r$ 颗糖果相邻配对，因此

$$\text{Cost}(x) = \sum_{i=1}^L a_i + \sum_{j=1}^r a_{L+2j}.$$

- ▶ 第二项只会取区间 $(L, x]$ 中与 x 奇偶性相同的位置。
- ▶ 分别维护普通前缀和 Pre、奇数位置和 Odd、偶数位置和 Even。
- ▶ 每个 x 的费用可以在 $\mathcal{O}(1)$ 时间内求出。
- ▶ 总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1 int T,n,k,Line[MAXN];
2 long long m,Pre[MAXN],Odd[MAXN],Even[MAXN];
3 long long Calc(int x){
4     int Count=min(k,x/2),Left=x-Count*2;
5     long long Ret=Pre[Left];
6     if(x&1) Ret+=Odd[x]-Odd[Left];
7     else Ret+=Even[x]-Even[Left];
8     return Ret;
9 }
```


正解 100 Pts

```
1 void Solve(){
2     scanf("%d%lld%d",&n,&m,&k);
3     for(int i=1;i<=n;i++) scanf("%d",&Line[i]);
4     sort(Line+1,Line+n+1);
5     Pre[0]=Odd[0]=Even[0]=0;
6     for(int i=1;i<=n;i++){
7         Pre[i]=Pre[i-1]+Line[i];
8         Odd[i]=Odd[i-1];Even[i]=Even[i-1];
9         if(i&1) Odd[i]+=Line[i];
```

正解 100 Pts

```
1      else Even[i]+=Line[i];
2  }
3  int Ans=0;
4  for(int i=1;i<=n;i++){
5      if(Calc(i)<=m) Ans=i;
6  }
7  printf("%d\n",Ans);
8  return;
9 }
```

Contents

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间

数据点 1 ~ 4 16 Pts

- 这一档保证 $n \leq 20$ 。枚举所有 2^n 个购买集合。

数据点 1 ~ 4 16 Pts

- ▶ 这一档保证 $n \leq 20$ 。枚举所有 2^n 个购买集合。
- ▶ 对每个集合计算清仓价格之和与原价之和，保留总价格不超过 m 的方案。

数据点 1 ~ 4 16 Pts

- ▶ 这一档保证 $n \leq 20$ 。枚举所有 2^n 个购买集合。
- ▶ 对每个集合计算清仓价格之和与原价之和，保留总价格不超过 m 的方案。
- ▶ 时间复杂度为 $\mathcal{O}(n2^n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 1 ~ 4 16 Pts

- ▶ 这一档保证 $n \leq 20$ 。枚举所有 2^n 个购买集合。
- ▶ 对每个集合计算清仓价格之和与原价之和，保留总价格不超过 m 的方案。
- ▶ 时间复杂度为 $\mathcal{O}(n2^n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 暴力的瓶颈在于同时决定两类物品的购买数量与具体选择。

数据点 5 ~ 8 32 Pts

- ▶ 这一档保证所有物品的清仓价格均为 1。

数据点 5 ~ 8 32 Pts

- ▶ 这一档保证所有物品的清仓价格均为 1。
- ▶ 至多购买 $\min(n, m)$ 件物品。因为每件物品占用的预算相同，应选择原价最大的若干件。

数据点 5 ~ 8 32 Pts

- ▶ 这一档保证所有物品的清仓价格均为 1。
- ▶ 至多购买 $\min(n, m)$ 件物品。因为每件物品占用的预算相同，应选择原价最大的若干件。
- ▶ 将原价从大到小排序后取前 $\min(n, m)$ 项之和即可。

数据点 5 ~ 8 32 Pts

- ▶ 这一档保证所有物品的清仓价格均为 1。
- ▶ 至多购买 $\min(n, m)$ 件物品。因为每件物品占用的预算相同，应选择原价最大的若干件。
- ▶ 将原价从大到小排序后取前 $\min(n, m)$ 项之和即可。
- ▶ 时间复杂度为 $\mathcal{O}(n \log n)$ 。

数据点 9 ~ 12 48 Pts

- ▶ 这一档保证所有物品的清仓价格均为 2。

数据点 9 ~ 12 48 Pts

- ▶ 这一档保证所有物品的清仓价格均为 2。
- ▶ 至多购买 $\min(n, \lfloor m/2 \rfloor)$ 件物品，同样选择原价最大的若干件。

数据点 9 ~ 12 48 Pts

- ▶ 这一档保证所有物品的清仓价格均为 2。
- ▶ 至多购买 $\min(n, \lfloor m/2 \rfloor)$ 件物品，同样选择原价最大的若干件。
- ▶ 可以发现，当同类物品的清仓价格相同时，只需要知道购买数量，具体选择一定是原价最大的前若干件。

数据点 13 ~ 19 76 Pts

- ▶ 将清仓价格为 1 与 2 的物品分别按原价从大到小排序。

数据点 13 ~ 19 76 Pts

- ▶ 将清仓价格为 1 与 2 的物品分别按原价从大到小排序。
- ▶ 枚举购买 x 件价格为 2 的物品，剩余预算为 $m - 2x$ ，再购买至多 $m - 2x$ 件价格为 1 的物品。

数据点 13 ~ 19 76 Pts

- ▶ 将清仓价格为 1 与 2 的物品分别按原价从大到小排序。
- ▶ 枚举购买 x 件价格为 2 的物品，剩余预算为 $m - 2x$ ，再购买至多 $m - 2x$ 件价格为 1 的物品。
- ▶ 直接累加每次选择的物品价值，时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 13 ~ 19 76 Pts

- ▶ 将清仓价格为 1 与 2 的物品分别按原价从大到小排序。
- ▶ 枚举购买 x 件价格为 2 的物品，剩余预算为 $m - 2x$ ，再购买至多 $m - 2x$ 件价格为 1 的物品。
- ▶ 直接累加每次选择的物品价值，时间复杂度为 $\mathcal{O}(n^2)$ 。
- ▶ 这一做法已经得到完整决策过程，瓶颈只是重复求两类物品的前若干项之和。

正解 100 Pts

- ▶ 分别记两类物品按原价降序排列后的序列为 A, B ，并维护前缀和 S_A, S_B 。

正解 100 Pts

- ▶ 分别记两类物品按原价降序排列后的序列为 A, B ，并维护前缀和 S_A, S_B 。
- ▶ 固定购买 x 件价格为 2 的物品时，选择 B 中前 x 件一定不劣。

正解 100 Pts

- ▶ 分别记两类物品按原价降序排列后的序列为 A, B ，并维护前缀和 S_A, S_B 。
- ▶ 固定购买 x 件价格为 2 的物品时，选择 B 中前 x 件一定不劣。
- ▶ 此时还能购买

$$y = \min(|A|, m - 2x)$$

件价格为 1 的物品，也应选择 A 中前 y 件。

正解 100 Pts

- ▶ 分别记两类物品按原价降序排列后的序列为 A, B ，并维护前缀和 S_A, S_B 。
- ▶ 固定购买 x 件价格为 2 的物品时，选择 B 中前 x 件一定不劣。
- ▶ 此时还能购买

$$y = \min(|A|, m - 2x)$$

件价格为 1 的物品，也应选择 A 中前 y 件。

- ▶ 当前方案的价值为 $S_B(x) + S_A(y)$ 。

正解 100 Pts

- 枚举所有满足 $0 \leq x \leq |B|$ 且 $2x \leq m$ 的整数 x ，取价值最大值。

正解 100 Pts

- ▶ 枚举所有满足 $0 \leq x \leq |B|$ 且 $2x \leq m$ 的整数 x ，取价值最大值。
- ▶ 任意合法方案都对应某个 x 。在这个 x 下，将两类物品分别替换成价值最大的同样件数，不会改变总价格，只会增大总价值。

正解 100 Pts

- ▶ 枚举所有满足 $0 \leq x \leq |B|$ 且 $2x \leq m$ 的整数 x ，取价值最大值。
- ▶ 任意合法方案都对应某个 x 。在这个 x 下，将两类物品分别替换成价值最大的同样件数，不会改变总价格，只会增大总价值。
- ▶ 因而枚举覆盖了某个最优方案，算法正确。

正解 100 Pts

- ▶ 枚举所有满足 $0 \leq x \leq |B|$ 且 $2x \leq m$ 的整数 x ，取价值最大值。
- ▶ 任意合法方案都对应某个 x 。在这个 x 下，将两类物品分别替换成价值最大的同样件数，不会改变总价格，只会增大总价值。
- ▶ 因而枚举覆盖了某个最优方案，算法正确。
- ▶ 排序复杂度为 $\mathcal{O}(n \log n)$ ，枚举复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- ▶ 枚举所有满足 $0 \leq x \leq |B|$ 且 $2x \leq m$ 的整数 x ，取价值最大值。
- ▶ 任意合法方案都对应某个 x 。在这个 x 下，将两类物品分别替换成价值最大的同样件数，不会改变总价格，只会增大总价值。
- ▶ 因而枚举覆盖了某个最优方案，算法正确。
- ▶ 排序复杂度为 $\mathcal{O}(n \log n)$ ，枚举复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 前缀和与答案都可能超过 `int`，需要使用 `long long`。

正解 100 Pts

```
1 int T,n,m,Value[MAXN],Type[MAXN],One[MAXN],Two[MAXN];
2 long long PreOne[MAXN],PreTwo[MAXN];
3 bool cmp(int a,int b){return a>b;}
4 void Solve(){
5     scanf("%d%d",&n,&m);
6     for(int i=1;i<=n;i++) scanf("%d",&Value[i]);
7     for(int i=1;i<=n;i++) scanf("%d",&Type[i]);
8     int CntOne=0,CntTwo=0;
9     for(int i=1;i<=n;i++){
10         if(Type[i]==1) One[++CntOne]=Value[i];
11         else Two[++CntTwo]=Value[i];
12     }
13     sort(One+1,One+CntOne+1,cmp);
14     sort(Two+1,Two+CntTwo+1,cmp);
15     PreOne[0]=PreTwo[0]=0;
16     for(int i=1;i<=CntOne;i++) PreOne[i]=PreOne[i-1]+One[i];
17     for(int i=1;i<=CntTwo;i++) PreTwo[i]=PreTwo[i-1]+Two[i];
```

正解 100 Pts

```
1  long long Ans=0;
2  for(int i=0;i<=CntTwo&& i*2<=m;i++){
3      int Count=min(CntOne,m-i*2);
4      Ans=max(Ans,PreTwo[i]+PreOne[Count]);
5  }
6  printf("%lld\n",Ans);
7  return;
8  }
```

Contents

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间

数据点 1 ~ 28 Pts

- ▶ 这一档保证 $n \leq 7$ 。枚举所有叶子的排列，并按照题意模拟探索过程。

数据点 1 ~ 28 Pts

- ▶ 这一档保证 $n \leq 7$ 。枚举所有叶子的排列，并按照题意模拟探索过程。
- ▶ 用标记数组维护已经被探索的结点。加入一个叶子时，从它向根跳父亲，直到遇到第一个已经被探索的结点。

数据点 1 ~ 28 Pts

- ▶ 这一档保证 $n \leq 7$ 。枚举所有叶子的排列，并按照题意模拟探索过程。
- ▶ 用标记数组维护已经被探索的结点。加入一个叶子时，从它向根跳父亲，直到遇到第一个已经被探索的结点。
- ▶ 由新结点的数量与其中最高结点的权值计算本次探索价值。

数据点 1 ~ 28 Pts

- ▶ 这一档保证 $n \leq 7$ 。枚举所有叶子的排列，并按照题意模拟探索过程。
- ▶ 用标记数组维护已经被探索的结点。加入一个叶子时，从它向根跳父亲，直到遇到第一个已经被探索的结点。
- ▶ 由新结点的数量与其中最高结点的权值计算本次探索价值。
- ▶ 若叶子数量为 k ，时间复杂度为 $\mathcal{O}(k!kn)$ 。

数据点 3 ~ 6 24 Pts

- ▶ 这一档可以对叶子集合进行状压 DP。

数据点 3 ~ 6 24 Pts

- ▶ 这一档可以对叶子集合进行状压 DP。
- ▶ 令 F_S 表示已经探索集合 S 中所有叶子时的最大价值。

数据点 3 ~ 6 24 Pts

- ▶ 这一档可以对叶子集合进行状压 DP。
- ▶ 令 F_S 表示已经探索集合 S 中所有叶子时的最大价值。
- ▶ 枚举下一个叶子 $v \notin S$ 。集合 S 对应的所有根到叶路径的并集已经确定，因此可以直接求出加入 v 时的新探索链。

数据点 3 ~ 6 24 Pts

- ▶ 这一档可以对叶子集合进行状压 DP。
- ▶ 令 F_S 表示已经探索集合 S 中所有叶子时的最大价值。
- ▶ 枚举下一个叶子 $v \notin S$ 。集合 S 对应的所有根到叶路径的并集已经确定，因此可以直接求出加入 v 时的新探索链。
- ▶ 转移为 $F_{S \cup \{v\}} \leftarrow \max(F_{S \cup \{v\}}, F_S + \text{Value}(S, v))$ 。

数据点 3 ~ 6 24 Pts

- ▶ 这一档可以对叶子集合进行状压 DP。
- ▶ 令 F_S 表示已经探索集合 S 中所有叶子时的最大价值。
- ▶ 枚举下一个叶子 $v \notin S$ 。集合 S 对应的所有根到叶路径的并集已经确定，因此可以直接求出加入 v 时的新探索链。
- ▶ 转移为 $F_{S \cup \{v\}} \leftarrow \max(F_{S \cup \{v\}}, F_S + \text{Value}(S, v))$ 。
- ▶ 这一做法仍然枚举了叶子的探索先后，但已经提示我们：真正影响价值的是每个结点第一次从哪个儿子方向被经过。

正解 100 Pts

- ▶ 一个探索序列会把所有结点恰好划分成若干条从上向下的链，每条链在它的最高结点处产生一次价值。

正解 100 Pts

- ▶ 一个探索序列会把所有结点恰好划分成若干条从上向下的链，每条链在它的最高结点处产生一次价值。
- ▶ 考虑一个非叶结点 u 。第一个在 u 子树内被探索的叶子只会属于一个儿子 v 。

正解 100 Pts

- ▶ 一个探索序列会把所有结点恰好划分成若干条从上向下的链，每条链在它的最高结点处产生一次价值。
- ▶ 考虑一个非叶结点 u 。第一个在 u 子树内被探索的叶子只会属于一个儿子 v 。
- ▶ 只有 v 中对应的链能够穿过 u 继续向上延伸。其余儿子的第一条链都从儿子本身开始，必须在儿子处结算。

正解 100 Pts

- ▶ 一个探索序列会把所有结点恰好划分成若干条从上向下的链，每条链在它的最高结点处产生一次价值。
- ▶ 考虑一个非叶结点 u 。第一个在 u 子树内被探索的叶子只会属于一个儿子 v 。
- ▶ 只有 v 中对应的链能够穿过 u 继续向上延伸。其余儿子的第一条链都从儿子本身开始，必须在儿子处结算。
- ▶ 因此处理完 u 的子树以后，只需要保留一条从 u 开始、尚未结算的链。

正解 100 Pts

定义 $d_{u,l}$ 表示:

- ▶ 已经确定 u 子树内的探索顺序。

正解 100 Pts

定义 $d_{u,l}$ 表示:

- ▶ 已经确定 u 子树内的探索顺序。
- ▶ 其中有一条从 u 开始、长度为 l 的链暂时不计算价值, 等待连接到 u 的祖先。

正解 100 Pts

定义 $d_{u,l}$ 表示:

- ▶ 已经确定 u 子树内的探索顺序。
- ▶ 其中有一条从 u 开始、长度为 l 的链暂时不计算价值, 等待连接到 u 的祖先。
- ▶ 除这条链以外, 其余新探索链均已结算时能够得到的最大总价值。

正解 100 Pts

定义 $d_{u,l}$ 表示:

- ▶ 已经确定 u 子树内的探索顺序。
- ▶ 其中有一条从 u 开始、长度为 l 的链暂时不计算价值, 等待连接到 u 的祖先。
- ▶ 除这条链以外, 其余新探索链均已结算时能够得到的最大总价值。

若 u 为叶子, 唯一的链只包含 u , 所以

$$d_{u,1} = 0,$$

其余状态均为负无穷。

正解 100 Pts

对于 u 的儿子 v ，先计算这棵子树完全在 v 处封口时的最优价值

$$g_v = \max_{1 \leq l \leq h_v} \{d_{v,l} + a_v b_l\}.$$

► $d_{v,l}$ 已经包含其他链的价值。

正解 100 Pts

对于 u 的儿子 v ，先计算这棵子树完全在 v 处封口时的最优价值

$$g_v = \max_{1 \leq l \leq h_v} \{d_{v,l} + a_v b_l\}.$$

- ▶ $d_{v,l}$ 已经包含其他链的价值。
- ▶ $a_v b_l$ 是保留链在最高结点 v 处产生的价值。

正解 100 Pts

对于 u 的儿子 v ，先计算这棵子树完全在 v 处封口时的最优价值

$$g_v = \max_{1 \leq l \leq h_v} \{d_{v,l} + a_v b_l\}.$$

- ▶ $d_{v,l}$ 已经包含其他链的价值。
- ▶ $a_v b_l$ 是保留链在最高结点 v 处产生的价值。
- ▶ 因而若 v 的链不向父亲延伸，它对父亲转移的完整贡献就是 g_v 。

正解 100 Pts

若选择儿子 v 的保留链穿过 u ，原来长度为 l 的链会增长为 $l+1$ 。其余儿子 w 的子树全部在 w 处封口，因此

$$d_{u,l+1} = \max_{v \in \text{son}(u)} \left\{ d_{v,l} + \sum_{w \in \text{son}(u)} g_w - g_v \right\}.$$

► 转移范围为 $1 \leq l \leq h_v$ 。

正解 100 Pts

若选择儿子 v 的保留链穿过 u ，原来长度为 l 的链会增长为 $l+1$ 。其余儿子 w 的子树全部在 w 处封口，因此

$$d_{u,l+1} = \max_{v \in \text{son}(u)} \left\{ d_{v,l} + \sum_{w \in \text{son}(u)} g_w - g_v \right\}.$$

- ▶ 转移范围为 $1 \leq l \leq h_v$ 。
- ▶ 所有不可达状态均为负无穷，不能用 0 代替，因为 a_u, b_l 可以为负数。

正解 100 Pts

- 根结点没有祖先，最后一条保留链也必须在根处结算，所以答案为

$$\max_{1 \leq l \leq h_1} \{d_{1,l} + a_1 b_l\}.$$

正解 100 Pts

- 根结点没有祖先，最后一条保留链也必须在根处结算，所以答案为

$$\max_{1 \leq l \leq h_1} \{d_{1,l} + a_1 b_l\}.$$

- 必要性：任意探索序列在每个非叶结点都只有第一个被探索的儿子方向能够向上延伸，因此一定对应某次转移。

正解 100 Pts

- ▶ 根结点没有祖先，最后一条保留链也必须在根处结算，所以答案为

$$\max_{1 \leq l \leq h_1} \{d_{1,l} + a_1 b_l\}.$$

- ▶ 必要性：任意探索序列在每个非叶结点都只有第一个被探索的儿子方向能够向上延伸，因此一定对应某次转移。
- ▶ 充分性：先探索被选中儿子中的保留链，再依次完成其余儿子的最优探索顺序，就能实现转移给出的链划分与价值。

正解 100 Pts

- ▶ 根结点没有祖先，最后一条保留链也必须在根处结算，所以答案为

$$\max_{1 \leq l \leq h_1} \{d_{1,l} + a_1 b_l\}.$$

- ▶ 必要性：任意探索序列在每个非叶结点都只有第一个被探索的儿子方向能够向上延伸，因此一定对应某次转移。
- ▶ 充分性：先探索被选中儿子中的保留链，再依次完成其余儿子的最优探索顺序，就能实现转移给出的链划分与价值。
- ▶ 因而 DP 恰好覆盖全部合法探索序列。

正解 100 Pts

- 记 h_u 为 u 子树的高度，也就是 $d_{u,l}$ 的有效长度上界。

正解 100 Pts

- ▶ 记 h_u 为 u 子树的高度，也就是 $d_{u,l}$ 的有效长度上界。
- ▶ 每个非根结点 v 的 DP 数组只会在处理父亲时被完整扫描常数次。

正解 100 Pts

- ▶ 记 h_u 为 u 子树的高度，也就是 $d_{u,l}$ 的有效长度上界。
- ▶ 每个非根结点 v 的 DP 数组只会在处理父亲时被完整扫描常数次。
- ▶ 因此总时间复杂度为

$$\mathcal{O} \left(\sum_{u=1}^n h_u \right).$$

正解 100 Pts

- ▶ 记 h_u 为 u 子树的高度，也就是 $d_{u,l}$ 的有效长度上界。
- ▶ 每个非根结点 v 的 DP 数组只会在处理父亲时被完整扫描常数次。
- ▶ 因此总时间复杂度为

$$\mathcal{O}\left(\sum_{u=1}^n h_u\right).$$

- ▶ 由于 $h_u \leq m$ ，有

$$\mathcal{O}\left(\sum_u h_u\right) \leq \mathcal{O}(nm).$$

正解 100 Pts

- ▶ 记 h_u 为 u 子树的高度, 也就是 $d_{u,l}$ 的有效长度上界。
- ▶ 每个非根结点 v 的 DP 数组只会在处理父亲时被完整扫描常数次。
- ▶ 因此总时间复杂度为

$$\mathcal{O}\left(\sum_{u=1}^n h_u\right).$$

- ▶ 由于 $h_u \leq m$, 有

$$\mathcal{O}\left(\sum_u h_u\right) \leq \mathcal{O}(nm).$$

- ▶ 空间复杂度同样可以做到 $\mathcal{O}(\sum_u h_u) \leq \mathcal{O}(nm)$, 在矮树上会明显优于直接保留完整的 $n \times m$ 数组。

正解 100 Pts

```
1  const long long INF=4e18;
2  int T,n,m,Fa[MAXN],Height[MAXN];
3  long long a[MAXN],b[MAXN],Dp[MAXN][MAXM],Best[MAXN];
4  vector<int> Son[MAXN];
5  void Solve(){
6      scanf("%d%d",&n,&m);
7      for(int i=1;i<=n;i++) Son[i].clear();
8      for(int i=2;i<=n;i++){
9          scanf("%d",&Fa[i]);
10         Son[Fa[i]].push_back(i);
11     }
12     for(int i=1;i<=n;i++) scanf("%lld",&a[i]);
13     for(int i=1;i<=n;i++) scanf("%lld",&b[i]);
14     memset(Dp,0x80,sizeof(Dp));
```

正解 100 Pts

```
1  for(int Now=n;Now>=1;Now--){
2      if(Son[Now].empty()){
3          Height[Now]=1;
4          Dp[Now][1]=0;
5          continue;
6      }
7      Height[Now]=1;
8      long long Sum=0;
9      for(int To:Son[Now]){
10         Height[Now]=max(Height[Now],Height[To]+1);
11         Best[To]=-INF;
12         for(int j=1;j<=Height[To];j++){
13             Best[To]=max(Best[To],Dp[To][j]+a[To]*b[j]);
14         }
15         Sum+=Best[To];
16     }
```

正解 100 Pts

```
1      for(int To:Son[Now]){
2          for(int j=1;j<=Height[To];j++){
3              Dp[Now][j+1]=max(Dp[Now][j+1],Dp[To][j]+Sum-Best[To]);
4          }
5      }
6  }
7  long long Ans=-INF;
8  for(int i=1;i<=Height[1];i++) Ans=max(Ans,Dp[1][i]+a[1]*b[i]);
9  printf("%lld\n",Ans);
10 return;
11 }
```


正解 100 Pts

```
1 int main(){
2     int c;
3     scanf("%d%d",&c,&T);
4     while(T-->0) Solve();
5     return 0;
6 }
```

Contents

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间

数据点 1 ~ 3 15 Pts

- 对每次询问枚举分割点 m 。

数据点 1 ~ 3 15 Pts

- ▶ 对每次询问枚举分割点 m 。
- ▶ 分别使用栈模拟左右两段的消除，再拼接两段剩余序列继续模拟。

数据点 1 ~ 3 15 Pts

- ▶ 对每次询问枚举分割点 m 。
- ▶ 分别使用栈模拟左右两段的消除，再拼接两段剩余序列继续模拟。
- ▶ 记录拼接以后新弹出的元素对数，并对所有分割点求和。

数据点 1 ~ 3 15 Pts

- ▶ 对每次询问枚举分割点 m 。
- ▶ 分别使用栈模拟左右两段的消除，再拼接两段剩余序列继续模拟。
- ▶ 记录拼接以后新弹出的元素对数，并对所有分割点求和。
- ▶ 直接实现的时间复杂度为 $\mathcal{O}(qn^2)$ 。

数据点 4 ~ 9 45 Pts

- ▶ 消除结束后的序列中不存在相邻且相同的元素，因此它一定是一个 0,1 交替序列。

数据点 4 ~ 9 45 Pts

- ▶ 消除结束后的序列中不存在相邻且相同的元素，因此它一定是一个 0,1 交替序列。
- ▶ 对每个区间，可以从左向右维护消除结果。加入一个元素时，结果长度只会增加 1 或减少 1。

数据点 4 ~ 9 45 Pts

- ▶ 消除结束后的序列中不存在相邻且相同的元素，因此它一定是一个 0,1 交替序列。
- ▶ 对每个区间，可以从左向右维护消除结果。加入一个元素时，结果长度只会增加 1 或减少 1。
- ▶ 预处理每个询问中所有前缀与后缀的消除长度，就能在 $\mathcal{O}(1)$ 时间计算一个分割点。

数据点 4 ~ 9 45 Pts

- ▶ 消除结束后的序列中不存在相邻且相同的元素，因此它一定是一个 0,1 交替序列。
- ▶ 对每个区间，可以从左向右维护消除结果。加入一个元素时，结果长度只会增加 1 或减少 1。
- ▶ 预处理每个询问中所有前缀与后缀的消除长度，就能在 $\mathcal{O}(1)$ 时间计算一个分割点。
- ▶ 单次询问复杂度降为 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(qn)$ 。

数据点 10 ~ 15 75 Pts

定义

$$s_i = \begin{cases} 1, & a_i \equiv i \pmod{2}, \\ -1, & a_i \not\equiv i \pmod{2}, \end{cases} \quad P_i = \sum_{j=1}^i s_j.$$

- 设当前被消去的两个元素来自原序列中的位置 $x < y$ 。二者之间的 $y - x - 1$ 个元素已经全部被成对消去，所以 x, y 的奇偶性相反。

数据点 10 ~ 15 75 Pts

定义

$$s_i = \begin{cases} 1, & a_i \equiv i \pmod{2}, \\ -1, & a_i \not\equiv i \pmod{2}, \end{cases} \quad P_i = \sum_{j=1}^i s_j.$$

- ▶ 设当前被消去的两个元素来自原序列中的位置 $x < y$ 。二者之间的 $y - x - 1$ 个元素已经全部被成对消去，所以 x, y 的奇偶性相反。
- ▶ 被消去的两个元素数值相同、下标奇偶性相反，故对应的 s_x, s_y 互为相反数。

数据点 10 ~ 15 75 Pts

定义

$$s_i = \begin{cases} 1, & a_i \equiv i \pmod{2}, \\ -1, & a_i \not\equiv i \pmod{2}, \end{cases} \quad P_i = \sum_{j=1}^i s_j.$$

- ▶ 设当前被消去的两个元素来自原序列中的位置 $x < y$ 。二者之间的 $y - x - 1$ 个元素已经全部被成对消去，所以 x, y 的奇偶性相反。
- ▶ 被消去的两个元素数值相同、下标奇偶性相反，故对应的 s_x, s_y 互为相反数。
- ▶ 因而区间内所有 s_i 的和在消除过程中保持不变。

数据点 10 ~ 15 75 Pts

- ▶ 最终相邻的两个元素之间同样已经消去了偶数个元素，故它们的原下标奇偶性相反。

数据点 10 ~ 15 75 Pts

- ▶ 最终相邻的两个元素之间同样已经消去了偶数个元素，故它们的原下标奇偶性相反。
- ▶ 最终序列中的元素交替出现。相邻元素的数值与下标奇偶性同时改变，所以对应的 s_i 相同。

数据点 10 ~ 15 75 Pts

- ▶ 最终相邻的两个元素之间同样已经消去了偶数个元素，故它们的原下标奇偶性相反。
- ▶ 最终序列中的元素交替出现。相邻元素的数值与下标奇偶性同时改变，所以对应的 s_i 相同。
- ▶ 因此所有剩余元素对应的 s_i 均同号，区间消除后的长度就是区间 s_i 之和的绝对值。

数据点 10 ~ 15 75 Pts

- ▶ 最终相邻的两个元素之间同样已经消去了偶数个元素，故它们的原下标奇偶性相反。
- ▶ 最终序列中的元素交替出现。相邻元素的数值与下标奇偶性同时改变，所以对应的 s_i 相同。
- ▶ 因此所有剩余元素对应的 s_i 均同号，区间消除后的长度就是区间 s_i 之和的绝对值。
- ▶ 对任意区间 $[l, r]$ ，有

$$|\text{red}(a_l, \dots, a_r)| = |P_r - P_{l-1}|.$$

数据点 10 ~ 15 75 Pts

- ▶ 最终相邻的两个元素之间同样已经消去了偶数个元素，故它们的原下标奇偶性相反。
- ▶ 最终序列中的元素交替出现。相邻元素的数值与下标奇偶性同时改变，所以对应的 s_i 相同。
- ▶ 因此所有剩余元素对应的 s_i 均同号，区间消除后的长度就是区间 s_i 之和的绝对值。
- ▶ 对任意区间 $[l, r]$ ，有

$$|\text{red}(a_l, \dots, a_r)| = |P_r - P_{l-1}|.$$

- ▶ 枚举分割点即可做到 $\mathcal{O}(qn)$ 。

正解 100 Pts

- ▶ 对任意两个序列 U, V ，处理完 U 后，栈中恰好留下 $\text{red}(U)$ ；继续把 V 送入同一个栈，与直接处理 $U \circ V$ 完全相同。

正解 100 Pts

- ▶ 对任意两个序列 U, V , 处理完 U 后, 栈中恰好留下 $\text{red}(U)$; 继续把 V 送入同一个栈, 与直接处理 $U \circ V$ 完全相同。
- ▶ 因此

$$\text{red}(\text{red}(U) \circ \text{red}(V)) = \text{red}(U \circ V).$$

正解 100 Pts

- ▶ 对任意两个序列 U, V ，处理完 U 后，栈中恰好留下 $\text{red}(U)$ ；继续把 V 送入同一个栈，与直接处理 $U \circ V$ 完全相同。

- ▶ 因此

$$\text{red}(\text{red}(U) \circ \text{red}(V)) = \text{red}(U \circ V).$$

- ▶ 所以先分别消除左右两段再拼接，不会改变整个区间的最终消除结果。

正解 100 Pts

对询问 $[l, r]$, 记

$$A = P_{l-1}, \quad B = P_r, \quad X = P_m.$$

分割点 m 左右两段消除后的长度分别为 $|X - A|$ 与 $|B - X|$, 两段拼接并继续消除后的长度等于整个区间的消除长度 $|B - A|$ 。

$$f(m) = \frac{|X - A| + |B - X| - |B - A|}{2}.$$

正解 100 Pts

定义

$$G(L, R, x) = \sum_{i=L}^R |P_i - x|.$$

将上一页的式子对 $l \leq m < r$ 求和, 可以得到

$$\text{Ans}(l, r) = \frac{G(l, r-1, P_{l-1}) + G(l, r-1, P_r) - (r-l)|P_r - P_{l-1}|}{2}.$$

► 每次原询问被拆成两个形如 $G(L, R, x)$ 的离线询问。

正解 100 Pts

定义

$$G(L, R, x) = \sum_{i=L}^R |P_i - x|.$$

将上一页的式子对 $l \leq m < r$ 求和, 可以得到

$$\text{Ans}(l, r) = \frac{G(l, r-1, P_{l-1}) + G(l, r-1, P_r) - (r-l)|P_r - P_{l-1}|}{2}.$$

- ▶ 每次原询问被拆成两个形如 $G(L, R, x)$ 的离线询问。
- ▶ 现在只需要快速求一个区间中所有 $|P_i - x|$ 的和。

正解 100 Pts

- ▶ 将位置 i 按 P_i 从小到大排序，将所有离线询问按 x 从小到大排序。

正解 100 Pts

- ▶ 将位置 i 按 P_i 从小到大排序，将所有离线询问按 x 从小到大排序。
- ▶ 扫描询问时，把所有满足 $P_i \leq x$ 的位置加入数据结构。

正解 100 Pts

- ▶ 将位置 i 按 P_i 从小到大排序，将所有离线询问按 x 从小到大排序。
- ▶ 扫描询问时，把所有满足 $P_i \leq x$ 的位置加入数据结构。
- ▶ 使用两棵树状数组，分别维护这些位置的数量与 P_i 之和。

正解 100 Pts

- ▶ 将位置 i 按 P_i 从小到大排序，将所有离线询问按 x 从小到大排序。
- ▶ 扫描询问时，把所有满足 $P_i \leq x$ 的位置加入数据结构。
- ▶ 使用两棵数状数组，分别维护这些位置的数量与 P_i 之和。
- ▶ 数状数组以下标位置为维度，因此能够查询任意区间 $[L, R]$ 中已经加入的数量与和。

正解 100 Pts

记区间内 $P_i \leq x$ 的数量与和为 $C_{\leq}, S_{\leq}$, 全部元素的数量与和为 C, S 。则

$$G(L, R, x) = xC_{\leq} - S_{\leq} \\ + (S - S_{\leq}) - x(C - C_{\leq}).$$

► 前两项统计不大于 x 的部分, 后两项统计大于 x 的部分。

正解 100 Pts

记区间内 $P_i \leq x$ 的数量与和为 $C_{\leq}, S_{\leq}$, 全部元素的数量与和为 C, S 。则

$$G(L, R, x) = xC_{\leq} - S_{\leq} \\ + (S - S_{\leq}) - x(C - C_{\leq}).$$

- ▶ 前两项统计不大于 x 的部分, 后两项统计大于 x 的部分。
- ▶ S 可以由 P_i 的普通前缀和求出。

正解 100 Pts

记区间内 $P_i \leq x$ 的数量与和为 $C_{\leq}, S_{\leq}$, 全部元素的数量与和为 C, S 。则

$$G(L, R, x) = xC_{\leq} - S_{\leq} \\ + (S - S_{\leq}) - x(C - C_{\leq}).$$

- ▶ 前两项统计不大于 x 的部分, 后两项统计大于 x 的部分。
- ▶ S 可以由 P_i 的普通前缀和求出。
- ▶ 单组时间复杂度为 $\mathcal{O}((n+q)\log n)$, 整份数据的时间复杂度为 $\mathcal{O}(\sum(n+q)\log n)$ 。

正解 100 Pts

记区间内 $P_i \leq x$ 的数量与和为 $C_{\leq}, S_{\leq}$, 全部元素的数量与和为 C, S 。则

$$G(L, R, x) = xC_{\leq} - S_{\leq} \\ + (S - S_{\leq}) - x(C - C_{\leq}).$$

- ▶ 前两项统计不大于 x 的部分, 后两项统计大于 x 的部分。
- ▶ S 可以由 P_i 的普通前缀和求出。
- ▶ 单组时间复杂度为 $\mathcal{O}((n+q) \log n)$, 整份数据的时间复杂度为 $\mathcal{O}(\sum (n+q) \log n)$ 。
- ▶ 单组空间复杂度为 $\mathcal{O}(n+q)$, 不同测试数据之间复用数组。

正解 100 Pts

```
1  int n,q,Line[MAXN],Pre[MAXN],Position[MAXN],Left[MAXN],Right[MAXN];
2  long long IndexSum[MAXN],Cnt[MAXN],Sum[MAXN],Part[MAXQ],Ans[MAXN];
3  struct ask{
4      int Left,Right,Value,Id;
5  }Ask[MAXQ];
6  bool cmpa(int x,int y){return Pre[x]<Pre[y];}
7  bool cmpb(ask x,ask y){return x.Value<y.Value;}
8  int lowbit(int x){return x&-x;}
9  void Modify(int x,long long Val,long long C[]){
10     for(;x<=n;x+=lowbit(x)) C[x]+=Val;
11 }
12 long long Query(int x,long long C[]){
13     long long Ret=0;
14     for(;x;x-=lowbit(x)) Ret+=C[x];
15     return Ret;
16 }
```

正解 100 Pts

```
1 void Solve(){
2     static int LastN=0;
3     scanf("%d%d",&n,&q);
4     int ClearN=max(n,LastN);
5     memset(Cnt,0,sizeof(long long)*(ClearN+1));
6     memset(Sum,0,sizeof(long long)*(ClearN+1));
7     Pre[0]=IndexSum[0]=0;
8     LastN=n;
9     for(int i=1;i<=n;i++){
10         scanf("%d",&Line[i]);
11         Pre[i]=Pre[i-1]+(Line[i]==(i&1)?1:-1);
12         IndexSum[i]=IndexSum[i-1]+Pre[i];
13         Position[i]=i;
14     }
```

正解 100 Pts

```
1  int Total=0;
2  for(int i=1;i<=q;i++){
3      scanf("%d%d",&Left[i],&Right[i]);
4      Ask[++Total]={Left[i],Right[i]-1,Pre[Left[i]-1],i*2-1};
5      Ask[++Total]={Left[i],Right[i]-1,Pre[Right[i]],i*2};
6  }
7  sort(Position+1,Position+n+1,cmpa);
8  sort(Ask+1,Ask+Total+1,cmpb);
```

正解 100 Pts

```
1  int Now=1;
2  for(int i=1;i<=Total;i++){
3      while(Now<=n&&Pre[Position[Now]]<=Ask[i].Value){
4          Modify(Position[Now],1,Cnt);
5          Modify(Position[Now],Pre[Position[Now]],Sum);
6          Now++;
7      }
8      int l=Ask[i].Left,r=Ask[i].Right;
9      long long CntLow=Query(r,Cnt)-Query(l-1,Cnt);
10     long long SumLow=Query(r,Sum)-Query(l-1,Sum);
11     long long Count=r-l+1,SumAll=IndexSum[r]-IndexSum[l-1];
12     Part[Ask[i].Id]=1ll*Ask[i].Value*CntLow-SumLow+
13                     SumAll-SumLow-1ll*Ask[i].Value*(Count-CntLow);
14 }
```

正解 100 Pts

```

1   for(int i=1;i<=q;i++){
2       Ans[i]=(Part[i*2-1]+Part[i*2]-
3           1ll*(Right[i]-Left[i])*abs(Pre[Right[i]]-Pre[Left[i]-1]))
4       )/2;
5       printf("%lld\n",Ans[i]);
6   }
7   int main(){
8       int c,T;
9       scanf("%d%d",&c,&T);
10      while(T-->0) Solve();
11      return 0;
12  }
```

Contents

T1 糖果店 (candy)

T2 清仓甩卖 (sale)

T3 树的价值 (tree)

T4 序列询问 (query)

吐槽时间